

# Databricks Execution Flow

**Repository:** Girish-LS/senior-de-assignment

End-to-end execution of the Databricks path for transaction ingestion, medallion-layer transformation, orchestration, and idempotent reruns.

REST API → Bronze → Silver → Gold

Notebook or external script

Delta + dbt + Databricks Job

## Verified repository evidence

- First bronze run: **352 fetched, 349 valid, 3 quarantined, 5 duplicate rows flagged across 5 duplicate groups**
- Incremental rerun: **17 re-read**, bronze row count stays **349**
- dbt on Databricks: **PASS=34 WARN=0 ERROR=0**
- Output artefacts committed under `outputs/databricks/`

### Bronze

Raw STRING-preserving  
Delta layer plus quarantine,  
watermark, and metrics.

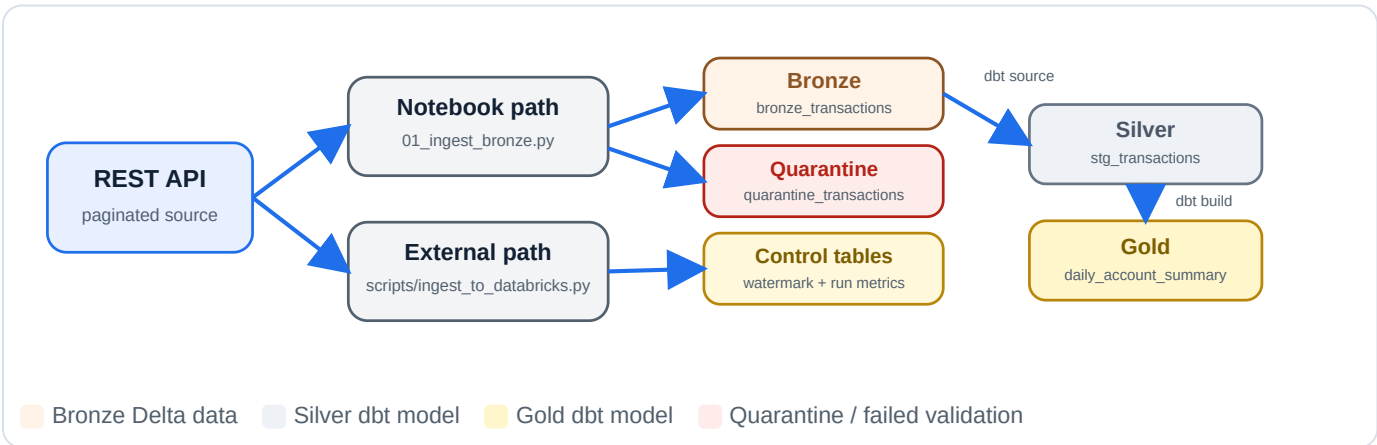
### Silver

stg\_transactions  
deduplicates survivors and  
casts types once.

### Gold

daily\_account\_summary  
incrementally recomputes  
account/day aggregates.

## 1. High-level architecture



## 2. Databricks object map

| Layer  | Object                                   | Contents                                                                                                                                               |
|--------|------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Bronze | workspace.bronze.bronze_transactions     | Validated source records persisted as Delta with every business field stored as STRING plus natural_key_hash, duplicate flags, and ingestion metadata. |
| Bronze | workspace.bronze.quarantine_transactions | Records that failed validation, including full raw payload, all failure reasons, error count, and failed fields.                                       |
| Bronze | workspace.bronze.pipeline_watermark      | Single-row high-water mark per source_system, computed from valid records only and written after a successful run.                                     |
| Bronze | workspace.bronze.ingestion_run_metrics   | One row per run: mode, status, fetched/valid/quarantined/duplicate counts, watermark before/after, duration, request stats.                            |
| Silver | workspace.silver.stg_transactions        | Deduplicated survivor rows with typed amount and transaction_day; the single definition of a usable transaction.                                       |
| Gold   | workspace.gold.daily_account_summary     | One row per account per UTC day with debit, credit, net, count, distinct merchants, top category, currencies, and update timestamp.                    |

### 3. Two execution paths write the same bronze layer

#### Notebook path (production-oriented)

##### File:

databricks/notebooks/01\_ingest\_bronze.py

- Runs on Databricks serverless compute.
- Reads credentials from secret scope transactions-api.
- Parameters: mode, catalog, bronze\_schema, secret\_scope, lookback\_hours.
- Imports the repository's own validation and dedupe code instead of reimplementing rules in the notebook.

```
dbutils.widgets.dropdown("mode",
"incremental", ["incremental", "full"])
dbutils.widgets.text("catalog", "workspace")
dbutils.widgets.text("bronze_schema",
"bronze")
dbutils.widgets.text("secret_scope",
"transactions-api")
dbutils.widgets.text("lookback_hours", "72")
```

#### External script path (workspace-light)

##### File:

scripts/ingest\_to\_databricks.py

- CLI execution from a workstation or automation shell.
- Uses databricks-sql-connector.
- Reads DATABRICKS\_HOST, DATABRICKS\_HTTP\_PATH, DATABRICKS\_TOKEN, and optional catalog/schema overrides.
- Writes the same four bronze objects and reuses the same pipeline components.

```
python
scripts/ingest_to_databricks.py --
mode full
python
scripts/ingest_to_databricks.py
# incremental
```

Both paths converge on the same storage design: Delta bronze tables in Unity Catalog followed by dbt-driven silver and gold models.

## 4. Step-by-step bronze execution flow

1. **Connect and create objects idempotently.** The Databricks path creates schema bronze and four Delta tables if they do not already exist.
2. **Decide full vs incremental mode.** Full load reads everything. Incremental mode reads the stored watermark and applies a 72-hour trailing lookback window.
3. **Fetch from the REST API.** Both Databricks endpoints use `TransactionsApiClient.iter_transactions()`.
4. **Validate each record.** Records either become typed `Transaction` objects or accumulate all validation failures.
5. **Flag duplicates.** Duplicate detection hashes all business fields except `transaction_id`; the lowest `transaction_id` survives.
6. **Write quarantine rows.** Failed records land in `quarantine_transactions`; a run clears its own quarantine rows first to stay idempotent within that run.
7. **Upsert bronze via MERGE.** Valid records are staged and merged into `bronze_transactions` on `transaction_id`.
8. **Advance watermark only after success.** The new watermark is the max valid `transaction_date` if it is higher than the previous value.
9. **Record metrics.** Each run writes a metrics row capturing counts, timing, and watermark movement.

### Watermark management

**Why gte instead of gt?** The API filter intentionally re-reads the boundary timestamp so records that share the exact watermark are not skipped forever.

```
effective_from = apply_lookback(watermark_before, 72)
# filter passed to the API: gte.<effective_from>
```

**Why compute the watermark from valid records only?** This dataset contains invalid records dated 2024-04-15 and 2024-11-31, later than every valid record. A raw-record watermark would jump to November and silently stop future ingestion.

**Zero-record guard.** A documented design decision for this repository's Databricks path is to treat a zero-row incremental lookback window as suspicious rather than healthy, because this path uses a `gte` boundary plus a trailing lookback window intended to include the watermark-setting record. The rationale is the historical malformed-filter case `gte.gte.<timestamp>`, which would otherwise appear as a successful but silently empty run.

### Validation and quarantine

- Validation enforces strict UTC timestamp format, positive decimal amount, accepted enumerations, non-blank merchant names, and a country-code membership check against the repository's ISO 3166-1 alpha-2 assigned-code set.
- Every failure reason is retained; validation does not stop at the first error.
- Quarantine rows store the raw JSON record, a semicolon-separated error message, error count, and failed field list.

```
quarantine_id, transaction_id, raw_record, error_reason, error_count,
failed_fields, ingestion_timestamp, ingestion_run_id, source_system
```

## Duplicate handling

- Duplicates are **flagged in bronze**, not dropped at ingestion time.
- The survivor is deterministic: the lowest `transaction_id` in each duplicate group.
- Silver resolves the correctness choice by filtering `is_duplicate = 0`.
- Committed evidence shows **5 duplicate groups** under `outputs/databricks/duplicate_groups.csv`.

## MERGE INTO upsert pattern

```

MERGE INTO workspace.bronze.bronze_transactions AS target
USING bronze_incoming AS source
  ON target.transaction_id = source.transaction_id
WHEN MATCHED      THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *

```

This is the Delta equivalent of the local SQLite `INSERT ... ON CONFLICT` pattern. It is the mechanism that makes the deliberate gte overlap safe.

## 5. dbt transformation path

### Silver: stg\_transactions

- Materialized as a **view**.
- Reads Databricks bronze as a dbt source whose schema is explicitly set to bronze.
- Casts amount once at the silver boundary.
- Creates `transaction_day` from the timestamp string.
- Filters out duplicate rows with where `is_duplicate = 0`.

```

select *
from {{ source('raw',
'bronze_transactions') }}
where is_duplicate = 0

```

### Gold: daily\_account\_summary

As currently defined in `dbt_project/models/marts/daily_account_summary.sql` and `dbt_project/models/marts/schema.yml`:

- `materialized='incremental'`, `unique_key=['account_id', 'transaction_date']`, `incremental_strategy='delete+insert'`, and `on_schema_change='fail'`.
- Includes only rows where `status = 'completed'`.
- Incremental predicate recomputes days where `transaction_day >= max(transaction_date) - interval '{{ var(\"summary_lookback_days\") }} days'`.
- The model contract is enforced in `dbt_project/models/marts/schema.yml`.

```

{{ config(
  materialized='incremental',
  unique_key=['account_id',
'transaction_date'],
  incremental_strategy='delete+insert'
) }}

```

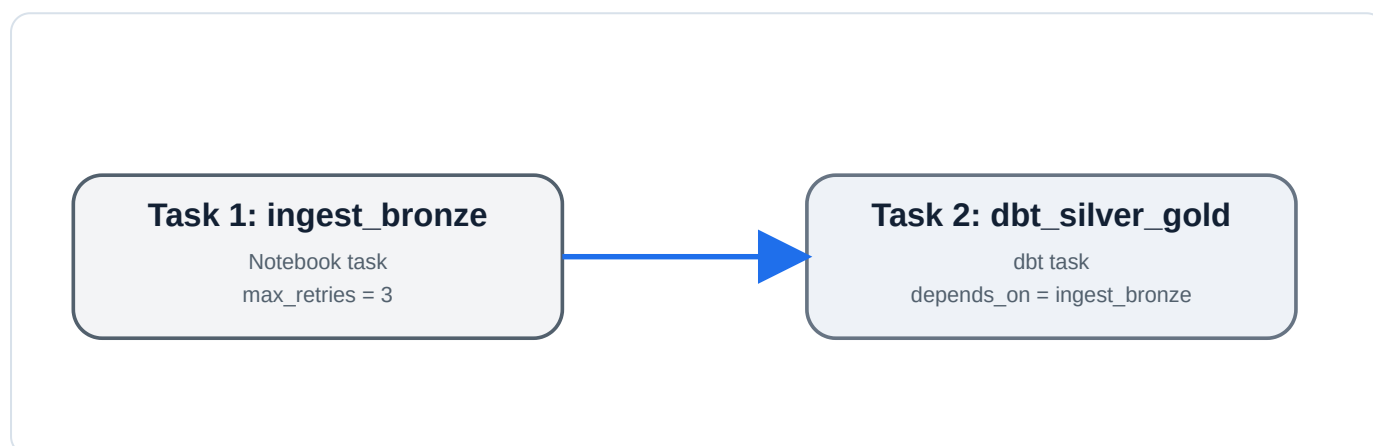
The exact Databricks build command is:

```
cd dbt_project
dbt deps
dbt build --target databricks
```

Committed output in `outputs/databricks/dbt_build.txt` shows **2 models** and **32 data tests**, with **1 exposure** declared in project metadata. The final execution status is **PASS=34 WARN=0 ERROR=0**, which counts the executed models and tests rather than the exposure declaration.

## 6. Orchestration job definition and task dependencies

See `databricks/jobs/transactions_pipeline_job.json` for the documented job-definition values. That existing repository file is referenced here but is not changed in this PR. The diagram below highlights the two-task dependency chain only.



| Task            | Type          | Retries                                                                                                                | Depends on                                        | Reasoning                                                                                                                   |
|-----------------|---------------|------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| ingest_bronze   | Notebook task | databricks/jobs/transactions_pipeline_job.json:<br>tasks[0].max_retries=3,<br>tasks[0].min_retry_interval_millis=60000 | —                                                 | API/network failures are often transient and should not force a full pipeline rerun.                                        |
| dbt_silver_gold | dbt task      | databricks/jobs/transactions_pipeline_job.json:<br>tasks[1].max_retries=1                                              | tasks[1].depends_on[0].task_key = 'ingest_bronze' | Failing data tests are usually not transient; retrying fetch plus transformation together would be wasteful and misleading. |

The referenced JSON file, which is outside this PR's diff, also documents scheduling, concurrency, notification, and job-health settings alongside the task definitions.

## 7. Portability findings from the DuckDB-first dbt path to Databricks

| Finding                               | What worked locally                                                                                | Databricks outcome / resolution                                                                                                                                                                                                                                                                         |
|---------------------------------------|----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| timestamp with time zone              | DuckDB contract accepted a timezone-aware timestamp type.                                          | Spark SQL does not expose that type in the same way. The checked-in resolution is visible in <code>dbt_project/models/marts/daily_account_summary.sql</code> and <code>dbt_project/models/marts/schema.yml</code> , which normalize the output to plain timestamp.                                      |
| VARCHAR in the dbt contract/cast path | DuckDB accepted the generated <code>cast(null as varchar)</code> shape used by the model contract. | That specific Databricks/Spark path raised <code>DATATYPE_MISSING_SIZE</code> ; the checked-in type change is reflected in <code>dbt_project/models/marts/daily_account_summary.sql</code> and <code>dbt_project/models/marts/schema.yml</code> , which now use string-compatible typing for this path. |
| <code>listagg order_by</code>         | DuckDB honors ordering for the currencies list.                                                    | The ongoing limitation is visible in <code>dbt_project/models/marts/daily_account_summary.sql</code> , and the Databricks warning is captured in <code>outputs/databricks/dbt_build.txt</code> : Spark/Databricks ignores <code>order_by_clause</code> in dbt's <code>listagg</code> .                  |
| UTF-8 BOM                             | DuckDB tolerated files that started with a BOM.                                                    | The issue is documented in <code>README.md</code> , and related BOM handling is visible in <code>ingestion/config.py</code> , which reads env files as <code>utf-8-sig</code> .                                                                                                                         |

The broader conclusion is that dbt model contracts are expressed in the destination warehouse's type system. Cross-engine portability therefore requires explicit output normalization, not just SQL that “looks portable”.

## 8. Key design decisions

- gte instead of gt:** prevents permanent loss of records that share the exact watermark timestamp.
- Watermark from valid rows only:** avoids the invalid-date trap created by later-dated quarantined records.
- Zero-record guard:** catches malformed incremental filters rather than treating an impossible empty window as success.
- Duplicates flagged in bronze, resolved in silver:** keeps source-quality observability while making the correctness choice reversible downstream.
- Bronze stores raw values as STRING:** preserves source fidelity and keeps money out of floating-point representation before silver decides types.
- Gold uses delete+insert, not merge:** aggregate rows must be recomputed from all contributing records when late-arriving data lands.

## 9. Committed evidence in outputs/databricks/

| File                      | What it proves                                                                                                                       |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| run_transcript.txt        | Full end-to-end execution transcript for full ingestion, incremental rerun, and dbt build.                                           |
| ingest_run1.json          | Initial full run: watermark was null, 352 fetched, 349 valid, 3 quarantined, and 5 duplicate rows flagged across 5 duplicate groups. |
| ingest_run2.json          | Incremental rerun: watermark before and after both 2024-03-30T21:01:36Z, 17 rows re-read from the lookback window.                   |
| bronze_sample.csv         | Sample raw-plus-metadata bronze rows written to Delta.                                                                               |
| quarantine_sample.csv     | Invalid records with all reasons preserved.                                                                                          |
| duplicate_groups.csv      | Five duplicate groups and their deterministic survivors.                                                                             |
| daily_account_summary.csv | Gold output produced from the Databricks path.                                                                                       |
| watermark.csv             | Current watermark state in the bronze control table.                                                                                 |
| run_metrics.csv           | Per-run operational history.                                                                                                         |
| idempotency_check.txt     | Explicit bronze count before vs after rerun: 349 → 349.                                                                              |
| table_counts.json         | Compact summary of table counts and current watermark.                                                                               |
| dbt_build.txt             | Databricks dbt success output: PASS=34 WARN=0 ERROR=0.                                                                               |

## 10. Idempotency proof

### Evidence from committed artefacts:

- ingest\_run1.json: watermark\_after = 2024-03-30T21:01:36Z
- ingest\_run2.json: same watermark\_before and watermark\_after, with effective\_filter\_from = 2024-03-27T21:01:36Z
- idempotency\_check.txt: bronze row count remains **349** before and after the rerun

```
bronze rows before rerun : 349
bronze rows after rerun  : 349
result                   : PASS - zero new rows
```

### Why the proof holds:

1. The incremental filter deliberately re-reads a trailing window using gte.
2. MERGE INTO matches on transaction\_id and updates existing bronze rows instead of duplicating them.
3. The watermark advances only after a successful write and only to the maximum valid transaction date.

The combination of those three choices makes incremental overlap safe, measurable, and repeatable.

## 11. Execution summary

The Databricks path is a medallion-style pipeline with two interchangeable ingestion entrypoints, a shared validation/deduplication core, Delta bronze control tables, dbt-modeled silver and gold layers, and a job

definition that keeps ingestion and transformation independently retryable. The design choices around gte, valid-only watermarking, and Delta MERGE INTO are not isolated implementation details; together they are the idempotency strategy.